94-844 Generative AI Lab

Prof. Beibei Li
Heinz College
Carnegie Mellon University

ASSIGNMENT 2 – Retrieval Augmented Generation (RAG) with LLMs

Overview: Retrieval Augmented Generation (RAG) is the process of creating and using custom data-sets as contextual information to include in the prompts submitted to LLMs (or in use with other types of ML models) to improve the quality of information provided to users in response to their queries. Building a high-quality RAG system involves multiple data and AI engineering steps. To develop performative AI engineering workflows from these steps, there are many design considerations that AI engineers and researchers need to take into account. These steps include:

RAG RECIPE

Ingredients:

(i) **Chunks:** Break up the text dataset (or other media) into smaller units of measurement *(in RAG engineering, this is called 'chunking').*

(ii) **Embeddings**: Transform the text chunks into numerical representations → these numerical representations are called ***embeddings***. These embeddings represent the contextual information provided in the text(s) attached as meta-data to the embeddings.

(iii) **Index**: Create an index for the database where you still store the embeddings (the so-called **vector store**). The index lists or groups the embeddings in a manner that enables searching for the embeddings most similar to the user's input embedding ('question').

(iv) **Vector Store:** Create a special database, called a **vector store**, that can handle storing ***high-dimensional data*** like embeddings. Part of this step includes selecting an **information retrieval algorithm** to use in the vector store. Algorithms commonly used in vector stores measure the **distance or similarity** of the embeddings, and some algorithms utilize **clustering** or **ranking,** too.

(vi) **Large Language Model (LLM)**: Integrate an LLM or machine learning model into this workflow at one or more points in the RAG architecture.

To achieve high-quality output from a RAG system, developers must experiment with the multiple ways of building a RAG. For example, how a text dataset is broken up into chunks (*e.g. units of measurement*) will affect what semantic meaning or knowledge that the embeddings for the text represent. In turn, the knowledge represented by embeddings impact what a user can retrieve from the vector store in a RAG system. For instance, breaking a text document up into words would enable the RAG to retrieve individual words most similar to those in a user question, whereas chunking the document semantically would enable the RAG to potentially retrieve data from the vector store that has a stronger semantic association with or relevance to the user question.

Assignment Objective: This assignment is designed to familiarize you with the different components and features of RAG that you need to conduct data-driven experiments to inform how to design the RAG system. Ultimately, the goal of this assignment is to give you a detailed understanding of how to conduct data-driven experiments to inform how you design a RAG system, and instill the confidence to use these data analytics skills in everyday practice.

To achieve the assignment objective, you will be provided with a dataset; you will need to choose a method of breaking up the dataset, creating embeddings from the chunks, and then uploading them to a vector store. After this, you will have the basic components of a RAG system built (minus an LLM or ML model). To help you learn how the different choices you've made in designing these system components might affect the quality of information retrieval, you will submit various queries to the vector store (e.g. embedding database), and analyze the relevance of the output from the vector store. Next, you will experiment with changing parameters in your user query submitted to the store, and analyze how that affects the quality of the output. After we will have you reconstruct the vector store after changing the way you break up the text dataset into units of measurement, as well, change the design of the vector store index and the meta-data attached to the embeddings in it. *What is the purpose of building these additional vector stores?* The purpose of doing this is to have you observe how your choice of 'chunking strategy' affects the output, and also, how the design of the index affects the information retrieval quality.

Optional Bonus Tasks: Optionally, you may earn extra points by completing bonus tasks that will have you utilize each of the main components of a RAG system (text chunking (and tokenizing); embeddings; vector store / similarity metrics; LLM generation; output evaluation) by crafting and changing the parameters of calls to LLMs, and evaluating the LLM responses.

Homework #2 – Required Tasks:

Part A. Experimenting with Queries in a Standard Vector Store Design (25 points)

Part A – Instructions:

1. **Data Selection & Engineering:**
    a. Upload the CMU Student Handbook to the Python notebook.
    b. Decide how you will 'chunk' the text data (e.g. at the word, sentence, paragraph level), and then chunk the CMU Handbook.
2. **Choose an embedding model.**
3. **Create embeddings for the text datasets** (CMU handbook).
4. **Create an index to use in the Vector Store** (follow the steps in the Python notebook provided in Lab tutorial video and Lab session).
5. **Load the index of embeddings into the Vector Store**.
6. **Experiment with Query Parameters & Queries in the Vector Store.**
    In this part of the homework, we will experiment with submitting pre-defined queries, provided below, to the vector store, and recording the responses to the queries.
    a. **Run Experiment:**
        i. **Requirement → submit the 5 queries below to the vector store**, and retrieve documents. Specifically, submit these 5 queries to the vector store – **setting the 'retrieval_number' parameter in the query function to 5**:
            a. **Query 1**: What is the academic integrity policy?
            b. **Query 2**: How is cheating defined in the policy?
            c. **Query 3**: What is the policy for improper or illegal communications?
            d. **Query 4**: What are CMU's quiet hours?
            e. **Query 5:** Where are pets or animals allowed on CMU's campus?
        ii. **Requirement → Record the Experiment Information**: Record the information retrieved from using the queries in this Google spreadsheet. **Submit this spreadsheet to Canvas along with your PDF responses to the homework questions**.
        https://docs.google.com/spreadsheets/d/1T2_fV0LKw9s_BEDKAvYZKKugDD0iBKSUlsnaui3Dmlg/edit?usp=sharing

7. **Requirement → Report Writing**: After recording your queries in responses in the Google spreadsheet, write responses to the homework questions, listed in this report template (using the report template is optional but providing responses to the questions is required): https://docs.google.com/document/d/1-TF9NjdrTxvkTDzK4BS2tSGuodKEvNClaEsWoPsfKMc/edit?usp=sharing

Part B. Experimenting with Retrievel_Number Parameter and Alternative Chunking and Embedding Strategies (25 points)

B.I. Select a Query from the List provided to you in Section A.

- **Requirement →** Choose 1 of the 5 queries in A.7, above, and use this query in the below experiments. In your homework submission, state which question you selected to use in the Part B experiments.

B.II. Retrieval_Number ("Top K") Parameter Experiment:

1. **Requirement → Conduct the following experiment:**
    a. Conduct an experiment by repeatedly submitting the query that you selected in the preceding step to the vector store; each time you submit the query, change the search parameter, in the following manner:
        i. 1st query submission to the vector store: set the **retrieval_number** parameter to be **retrieval_number**=1.
        ii. 2nd query submission to the vector store: set the **retrieval_number** parameter to be **retrieval_number** = 3
        iii. 3rd query submission to the vector store: set the **retrieval_number** parameter to be **retrieval_number** = 5
        iv. 4th query submission to the vector store: set the **retrieval_number** parameter to be **retrieval_number** = 10

    b. **Requirement → Record the *unique* responses to each query in this spreadsheet:** *(you do **not** need to record duplicate output. For example, when you change* **retrieval_number***=1 to* **retrieval_number***=3, one of the responses should be the same as the response you got when you set k=1)*

B.III. Chunking Strategy Experiment:

2. **Requirement → Utilize a Different Chunking Method & Conduct the following experiment:**
    a. Return to the prior steps for developing the vector store, to the chunking step, which comes before transforming the chunks into embeddings. To meet the homework requirements, now select and use a different chunking method (e.g. word, sentence, paragraph) for your dataset. You can choose any chunking method as long as it is not the same method you used previously. Specifically, follow these steps:
        b. Chunk your text data using the method.
        a. Create embeddings of the chunks.
        b. Create the **index** for the vector store.
        c. Load the embeddings into the vector store.
    d. Submit the query you selected in B.1 to the vector store 4 different times (using the different '**retrieval_number**' parameter settings defined in B.2 (i-iv), e.g. the preceding homework question). **Record the *unique* responses to each query in this spreadsheet:** https://docs.google.com/spreadsheets/d/1T2_fV0LKw9s_BEDKAvYZKKugDD0iBKSUlsnaui3Dmlg/edit?gid=0#gid=0

B.III. Index Strategy Experiment:

3. **Requirement → Utilize a Different Index Strategy & Conduct the following experiment:** Return to the prior steps for developing the vector store, to the **index creation step**. To meet the homework requirements, then follow these steps:
    a. Get the '**Policy Question' Dataset** from Canvas (published on Canvas as a CSV file named 'PolicyQuestions.csv') and load the dataset into the Jupyter/Python notebook (there is code in the notebook for doing this).
    b. Convert these 'policy' questions to embeddings.
    c. Attach the text chunks you created in Experiment 2 to these questions. Specifically, follow these steps:
        1. Attach the text chunks you created for Experiment 2 to these embeddings as metadata, following the steps in the Python Notebook.
        2. Create the vector store index, using the 'policy question' embeddings.
        3. Load the embeddings and metadata into the vector store.
    e. Once your vector store is created with this new index, submit the same query you selected in B.1 to the vector store 4 different times (using the

different '**retrieval_number**' parameter settings defined in B.2 (i-iv), e.g. the preceding homework question). **Record the *unique* responses to each query in this spreadsheet:** https://docs.google.com/spreadsheets/d/1T2_fV0LKw9s_BEDKAvYZKKugDD0iBKSUlsnaui3Dmlg/edit?gid=0#gid=0

Part C. Evaluating the Performance of Different Vector Stores with an LLM Judge Agent (25 points)

In this section, you'll create and use an LLM Judge to measure a characteristic of the information retrieved from the vector store with respect to the query you chose for your experiment. Below, you'll find instructions for creating the LLM Judge (and selecting a concept or characteristic it should measure).

Instructions / Guidelines for the LLM Judge:

1. Use an LLM Judge to prompt another LLM to evaluate the output from the vector store. To do this, you may either:
    a. Design your own LLM Judge, for example, by writing a system prompt;
    b. Use the LLM Judge in the Python Notebook;
    c. Or writing a similar Python function, such as with Pydantic or Dataclasses.

For the vector store responses you retrieved, from each of the three (3) vector stores you created:

1. **Score the Vector Store Responses**: Using the LLM Judge, score the responses (information) retrieved from each vector store.
2. **Performance & Information Retrieval Quality Metrics**: Produce and report performance and information retrieval quality metrics in your written report. Specifically, use the LLM Judge scores to produce these metrics:
    - **Search Time**: Search time measures how long it takes the system to produce information. In the Python notebook, it is measured by subtracting the start-time from the end-time in a given process.

    - **LLM Judge Precision Score**: Precision is a measure of accuracy. It is taken by computing the ratio of true positives to all possible positives (e.g. true + false positives). The equation for precision is:

      Precision = True Positive (TP) / (True Positives (TP) + False Positives (FP))

    - **LLM Judge Recall Score:** Recall is a measure of how well a system can produce relevant information. The equation for recall is:

Recall = True Positives (TP) / (True Positives (TP) + False Negatives (FP))

3. **Average Cost Metric:**
    - Lookup the per input token price and per output token price of the LLM model you used for the LLM Judge.
    - Compute and record the cost of each search that was performed for this homework assignment.
    - For each of the three (3) vector stores, compute the average cost of the searches. Report the average cost in your written report.

*** In the lab, and the recorded demo, your teaching team provides a demonstration of how to produce these metrics. Please email the teaching team with questions.**

Part D. Written Report & Presentation (25 points)

Written Report Requirement:

- **[Required]** Describe each of your experiments and their results in a written report; include your methodology, findings, insights, conclusions and responses to the homework questions for each experiment.
    - Please include answers to the **[required]** Homework Questions, listed here: https://docs.google.com/document/d/1-TF9NjdrTxvkTDzK4BS2tSGuodKEvNClaEsWoPsfKMc/edit?usp=sharing
- Submit your document to Canvas as a PDF.

3-5 Presentation Slides Requirement: **[Required]** With your team, prepare 3-5 slides. These slides should present highlights of your experimental hypotheses ('what information and what quality of information did you expect to retrieve from the vector store'?); your methodology ('how did you chunk the CMU Policy Document?', 'How did you design your LLM Judge?', etc.); your results ('What did you observe during your experiments?', 'What do your metrics suggest?', etc.); the implications of your metrics ('How might you change the design of the vector store?' 'What evidence do your results provide for these changes?'); and your recommendations ('What next steps or best practices would you recommend and why?').

- Submit your document to Canvas as a PDF.

Homework Technical / Code Instructions & Materials:

1. **Homework Assignment Instructions:** https://docs.google.com/document/d/1zoobmJ-Vw8F9J_PsyDMhTYOrbeFVAJtrTOlOAEz_7DQ/edit?usp=sharing
    a. **Homework Questions:** https://docs.google.com/document/d/1-TF9NjdrTxvkTDzK4BS2tSGuodKEvNClaEsWoPsfKMc/edit?usp=sharing

2. **Technical Instructions**: https://docs.google.com/document/d/1o5vYONxVZdabxnaMVoBhUt6o8qeOkwb-OR5ucSX7Bl0/edit?usp=sharing
3. **Python / Jupyter Notebook**: https://colab.research.google.com/drive/1VqwTbC6bwI5NAmLZSb_qUr25B57ABkf4?usp=sharing
4. **Datasets:**
    a. **CMU Policy Handbook:** https://drive.google.com/file/d/1dXNqYm9eqWsZxrl1qqjFZYP6fZd2QOGi/view?usp=drive_link
    b. **Policy Questions Dataset:** https://docs.google.com/spreadsheets/d/1qdCPSgs6XvkjEiYOyQ3G6Dq9qfRkn-occtguwEA6-MI/edit?usp=drive_link

Requirements for your homework submission:

1. **[Required] Homework Submission Report**: You are required to write high-quality written responses to the **[required]** homework questions and describe your experiments. Please submit this report to Canvas as PDF.
    a. **Homework Questions:** **https://docs.google.com/document/d/1-TF9NjdrTxvkTDzK4BS2tSGuodKEvNClaEsWoPsfKMc/edit?usp=sharing**

2. **[Required] Google Spreadsheet of Experiment Records**: You are required to copy this Google spreadsheet template and provide copies of the responses to your queries that were submitted to the vector store. https://docs.google.com/spreadsheets/d/1T2_fV0LKw9s_BEDKAvYZKKugDD0iBKSUlsnaui3Dmlg/edit?gid=0#gid=0
    a. *Please make sure to grant **Professor Li**, and TAs access to your Google document *before* submitting it.*
3. **[Required] Presentation Sides**: You are required to create 3-5 presentation slides. Please share a copy of your slides with the teaching team.

Grading

Grade Component	Weight of Total Grade 20%
Part A) Experimenting with Vector Store Query Design	5%

Part B) Experimenting with the Vector Store Embeddings & Query Parameters	5%
Part C) Evaluating the Performance of Different Vector Stores	5%
Part D) Written Report of Vector Store Experiments & Evaluation	5%

Grade Rubrics	**Share of Assignment**
Accuracy: – Have you followed the instructions correctly? – Have you understood the question correctly?	10%
Experimental Design & Execution: – Creativity and novelty in conducting and analyzing the design of the experiment.	20%
Completeness: – Have you answered each question (and subquestion)?	30%
Analytics & Insights: – Have you provided a clear written rationale and informative explanations for your choices and observations? While we don't require a certain length response, we encourage you to write high-quality responses to the questions asking you to explain your reasoning or analysis of the experiments. A high-quality response is subjective but informed by observations of your experimental results; it may also be informed by facts given in the literature or engineering documents, or other authoritative resources. Grammatical correctness is expected but not as important as sharing your opinion and reasoning about the experiments. There are not necessarily 'right' or 'wrong' answers.	30%
Scientific Rigor: – Documentation, and reproducibility of the results.	10%

Homework Submission FAQs:

1. ***Do I need to write any of my own Python code?***
   No. You will be provided with a Jupyter/Python notebook that you may use to complete the assignment. You are welcome to modify this notebook.

2. ***Am I required to submit any Python code or the Python notebook I use for the assignment?***
   Yes. You are required to submit the code you used for your LLM Judge.

3. ***What if the information retrieved from the vector store is low quality – will this affect my grade?***
   No. We are not grading you based on the quality of the output retrieved from the vector store. You will be graded on:

   - Carefully following the homework instructions – which include copying/pasting the output retrieved from the vector store into a Google spreadsheet (provided below and on Canvas).
   - The quality of your written analyses to the homework questions, which you will provide in a homework submission report. In your analyses, our hope is you will display strong reasoning skills in thinking about how different features of the RAG design might affect the quality of information retrieval.

4. ***What is the required submission format for the homework assignment?***
   You are required to submit a PDF of your written responses to the homework questions. *Please make sure to use the Homework Submission Report Template and to include all the information requested or listed in the template: https://docs.google.com/document/d/1-TF9NjdrTxvkTDzK4BS2tSGuodKEvNClaEsWoPsfKMc/edit?usp=sharing*

Appendix.

I. Homework 2 – Optional Reading List

Design Papers:

- **[Design]** David Rau, Shuai Wang, Hervé Déjean, Stéphane Clinchant, and Jaap Kamps. 2025. Context Embeddings for Efficient Answer Generation in Retrieval-Augmented Generation. In Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25). Association for Computing Machinery, New York, NY, USA, 493–502. https://doi.org/10.1145/3701551.3703527

- **[Design]** QuIM-RAG: Advancing Retrieval-Augmented Generation With Inverted Question Matching for Enhanced QA Performance.

- **[Design]** Enhancing RAG Pipeline Performance with Translation-Based Embedding Strategies for Non-English Documents.

Evaluation Papers:

- **[Evaluation]** Xiao Yang, et al. 2024. CRAG – Comprehensive RAG Benchmark. 38th Conference on Neural Information Processing Systems (NeurIPS 2024) Track on Datasets and Benchmarks.

- **[Evaluation]** Shuting Wang, et al. OmniEval: An Omnidirectional and Automatic RAG Evaluation Benchmark in Financial Domain. In the Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pages 5726–5751: November 4-9, Association for Computational Linguistics.

- **[Evaluation]** Kaige Xie, et al. Do RAG Systems Cover What Matters? Evaluating and Optimizing Responses with Sub-Question Coverage. Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), pages 5836–5849: April 29 - May 4, 2025, Association for Computational Linguistics.

- **[Evaluation]** Ronak Pradeep, Nandan Thakur, Shivani Upadhyay, Daniel Campos, Nick Craswell, Ian Soboroff, Hoa Trang Dang, and Jimmy Lin. 2025. The Great Nugget Recall: Automating Fact Extraction and RAG Evaluation with Large Language Models. In Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25), July 13–18, 2025, Padua, Italy. ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3726302.3730090

- **[Evaluation]** Lorenz Brehme, et al. Can LLMs Be Trusted for Evaluating RAG Systems? A Survey of Methods and Datasets. 2025 IEEE Swiss Conference on Data Science (SDS).

Safety / Security Papers:

- **[Safety & Security]** Shangeetha Sivasothy, et al. RAGProbe: Breaking RAG Pipelines with Evaluation Scenarios. 2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN).